Soham Chainani
D20A 49

EXPERIMENT 6

Aim:

Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.

THEORY

1. Smart Contract

A smart contract is a program deployed on a blockchain that automatically executes predefined rules when its
functions are called. In this experiment, a Ballot/Voting smart contract is created using Solidity and deployed using
Remix IDE.

The Ballot contract allows:

· The chairperson to grant voting rights to voters.
· Voters to cast their votes.
· Voters to delegate theIt is used to ensure that only authorized users can perform operations and to prevent invalid transactions. Thus, it
improves the security and reliability of smart contracts.

3. Mapping

A mapping is a Solidity data structure that associates keys with values, similar to a hash table. In the Ballot contract,
mapping connects Ethereum addresses with their corresponding voter information.

The voter information includes voting weight, voting status, delegated address and proposal index. Mappings
provide efficient data lookup but do not have a length property and cannot be directly iterated over.

4. Storage

Storage refers to permanent data stored on the blockchain. Variables stored in storage remain available across
transactions until they are modified.

In the Ballot contract, voter information is maintained in storage throughout the contract's lifecycle. However,
storing data on the blockchain consumes gas, making storage more expensive.ir votes to other voters.
. The contract to calculate the winning proposal.
. The name of the winning proposal to be displayed.

2. Require Statement

The require statement is used as a guard condition in Solidity. It checks whether a particular condition is true
before allowing a function to execute.

If the condition is false, the function execution stops and the transaction is reverted. Any changes made during that
transaction are rolled back.

5. Memory

Memory is temporary data storage used during function execution. It is mainly used for temporary variables, function arguments and computations.

The data stored in memory is discarded once the function execution is completed. Compared to storage, memory is less expensive and does not permanently store information on the blockchain.

6. Why bytes32 Instead of string?

`bytes32` is a fixed-size data type that stores exactly 32 bytes of data. It provides efficient storage, faster comparisons and lower gas consumption. However, it limits the proposal name to 32 bytes.

On the other hand, `string` is a dynamically sized data type that supports variable-length text. It is more flexible and readable but may require more gas and complex handling.

Therefore, `bytes32` is preferred when gas efficiency is important, whereas `string` is preferred for flexibility and readability.

7. Ballot Contract

The Ballot contract implements a voting mechanism using Solidity. It manages voters, voting rights, vote delegation and proposal selection. It also calculates the winning proposal based on the total votes received and displays the winner's name.

Output & Code:

Structure of Voter and Proposal

```
struct Voter {
uint weight;
bool voted;
address delegate;
uint vote;

struct Proposal {
string name;
uint voteCount;
```

Constructor

```
constructor(string[] memory proposalNames) {
chairperson = msg.sender;
voters[chairperson].weight = 1;

for (uint i=0; i< proposalNames.length; i++) {
proposals.push(Proposal({
name: proposalNames[i],
voteCount: 0
```

Give Right to Vote

```
function giveRightTo Vote(address voter) external {
require(msg.sender -- chairperson,
"Only chairperson can give right to vote.");

require(!voters[voter].voted,
"The voter already voted.");

require(voters[voter].weight == 0,
"Voter already has the right to vote.");

voters[voter].weight = 1;
```

Vote Delegation

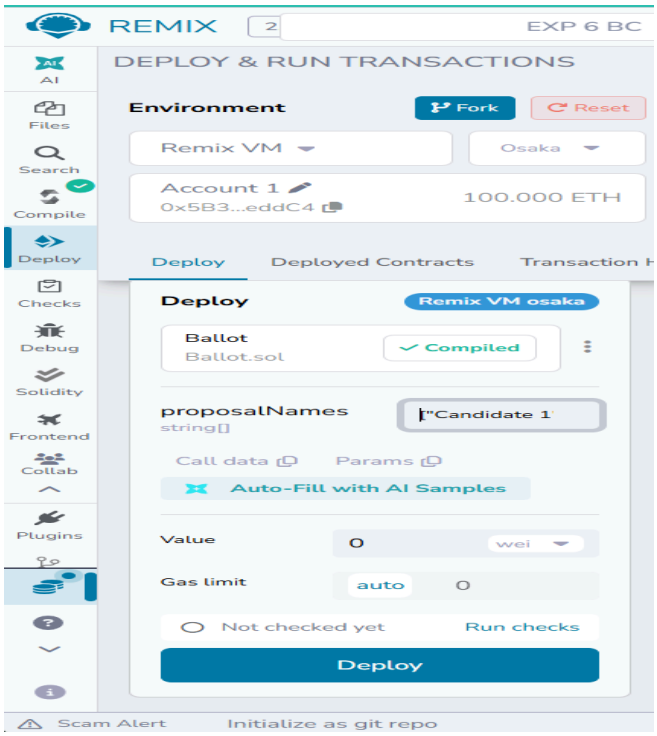
```
function delegate(address to) external {
Voter storage sender - voters[msg.sender];

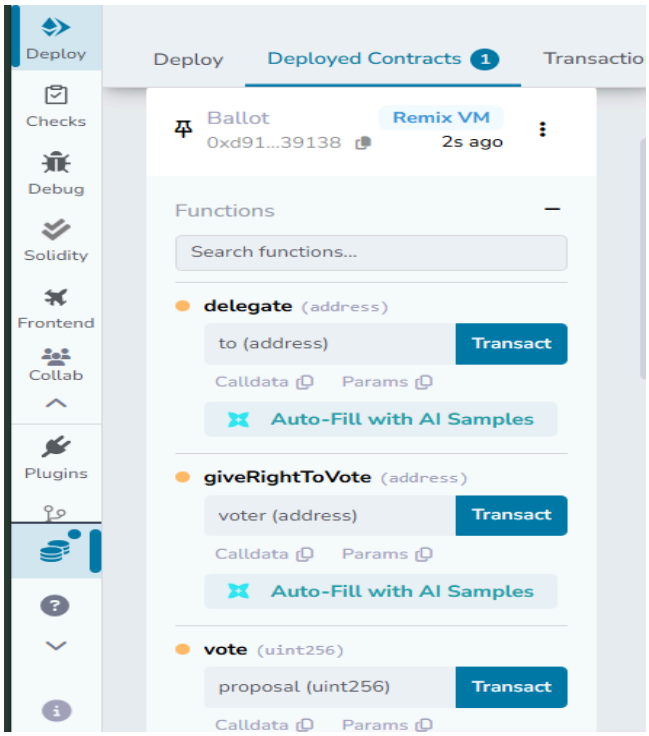
require(sender.weight != 0, "You have no right to vote");
require(!sender.voted, "You already voted.");
require(to != msg.sender, "Self-delegation is disallowed.");

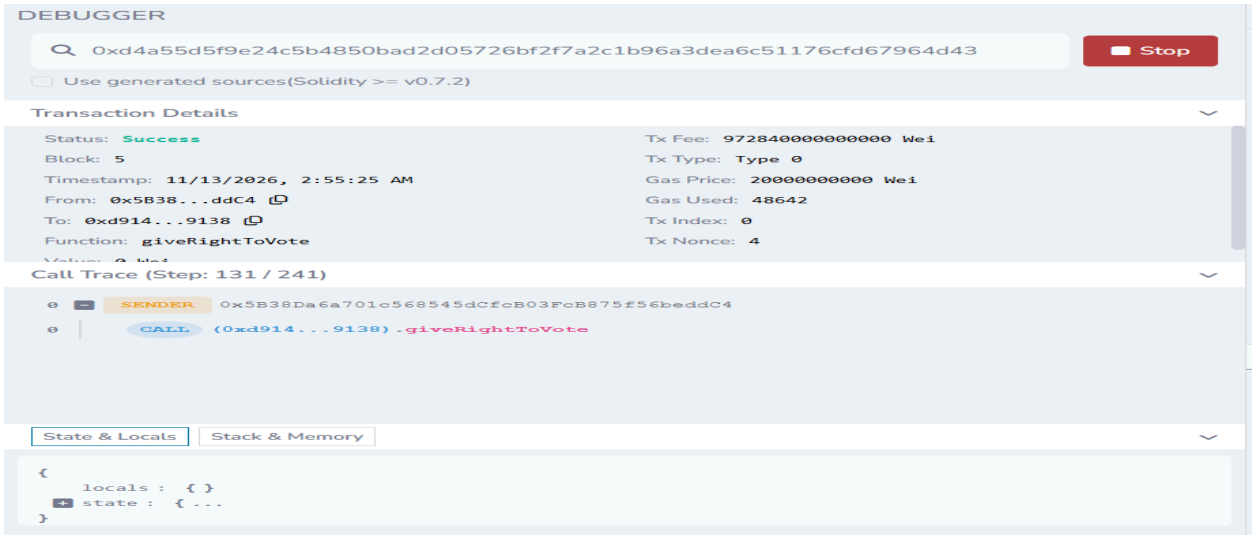
Voter storage delegate = voters[to];
require(delegate _. weight>=1);
```

```
sender.voted = true;
sender.delegate = to;

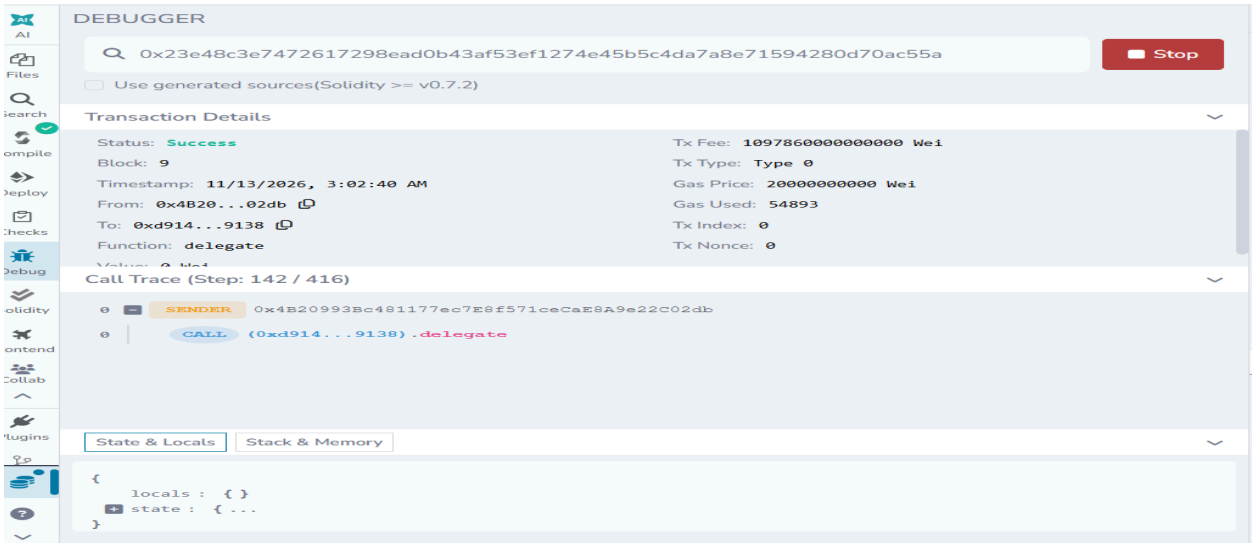
if (delegate _. voted) {
proposals[delegate _. vote].votcCount +- sender.weight;
} else {
delegate _. weight += sender.weight;
```











Conclusion

Successfully developed and deployed an ETH Transfer smart contract using Solidity on Remix IDE.
MetaMask was integrated to perform ETH transactions on the Sepolia Testnet. The transfer was successfully executed and verified using Sepolia Etherscan. Thus, the working of smart contracts and
blockchain transactions were understood.